VeriFACT AI: Real-Time LLM Hallucination Detection.

via Hybrid Retrieval and DeBERTa-based NLI

Under the Supervision of

Dr. Pushpendra Kumar

Submitted By

Aditya Ashish
Regd. No.: 22190503004

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CENTRAL UNIVERSITY OF JAHRKHAND

RANCHI, JHARKHAND 835222

Academic Year 20252026

Certificate

Name: Aditya Ashish
Registration Number: 22190503004
Title of the Project: VeriFACT AI: Real-Time LLM Hallucination Detection Using Retrieval-Augmented Verification

This is to certify that the project report entitled *"VeriFACT AI: Real-Time LLM Hallucination Detection via Hybrid Retrieval and DeBERTa-based NLI"* submitted by **Aditya Ashish** (Regd. No. 22190503004) of **Computer Science and Engineering**, Central University of Jharkhand, Ranchi, Jharkhand has been examined by us. We are satisfied with the volume, quality, and correctness of the work.

HOD, Dept. of CSE

Declaration

I hereby declare that the project work entitled **"VeriFACT AI: Real-Time LLM Hallucination Detection Using Retrieval-Augmented Verification"** is an authentic record of my own work carried out during the academic year 2025–2026 under the supervision of the Department of Computer Science and Engineering, Central University of Jharkhand, Ranchi. The matter presented in this report has not been submitted for the award of any other degree or diploma in any university or institution.

Aditya Ashish
B.Tech CSE (Final Year)
Central University of Jharkhand

Date: _____

Acknowledgment

I express my sincere gratitude to the faculty of the Department of Computer Science and Engineering, Central University of Jharkhand, for their guidance throughout this project. I thank the open-source communities behind HuggingFace Transformers, FAISS, spaCy, sentence-transformers, Ollama, and Groq for providing the foundational tools that made this work possible. I also acknowledge the researchers whose published work—particularly on FEVER [1], Retrieval-Augmented Generation [2], DeBERTa [3], TruthfulQA [4], SelfCheckGPT [7], and Semantic Entropy [9]—directly shaped the pipeline architecture.

Aditya Ashish

Abstract

Large Language Models (LLMs) such as ChatGPT, Claude, and Gemini generate fluent but sometimes factually incorrect text—a phenomenon known as hallucination, occurring in 3–27% of outputs [4]. Existing detection tools (SelfCheckGPT, FActScore, SAFE) operate offline and lack real-time, user-facing interfaces.

This report presents VERIFACT AI, a real-time hallucination detection system that operates as a Chrome browser extension on ChatGPT, Claude, Gemini, and Grok. The system employs a six-stage verification pipeline: (1) claim decomposition via Groq Llama 3.1 70B or spaCy fallback, (2) rule-based contradiction check against 205 hardcoded facts, (3) hybrid evidence retrieval combining a local FAISS vector index (11,248 chunks) with live Wikipedia API access (6.8 million articles), BM25 sparse search, and cross-encoder reranking, (4) Natural Language Inference using DeBERTa-v3-base with a specificity gate and fabrication detection, (5) SelfCheck consistency scoring with semantic entropy, and (6) Bayesian five-signal confidence fusion.

The system achieves 93.8% accuracy on the HaluEval benchmark (96.6% F1), 100% accuracy on rule-engine-covered claims, and detects fabricated entities (e.g., fictitious theorems and protocols) via exact-title Wikipedia verification. The entire system operates at zero cost using HuggingFace Spaces (free CPU), Groq API (free LLM), Wikipedia API (free knowledge), and Vercel (free hosting).

Keywords: Hallucination Detection, Retrieval-Augmented Generation, Natural Language Inference, Browser Extension, DeBERTa, FAISS, Real-Time Verification, Fabrication Detection

Contents

1 Introduction **9**
- 1.1 Objective . . . 9
- 1.2 Motivation . . . 9
- 1.3 Project Scope . . . 9
- 1.4 Report Organization . . . 10
- 1.5 Literature Survey . . . 10

2 System Architecture and Methodology **12**
- 2.1 High-Level Architecture . . . 12
- 2.2 The Six-Stage Verification Pipeline . . . 13
  - 2.2.1 Stage 1: Claim Decomposition . . . 13
  - 2.2.2 Stage 2a: Rule-Based Contradiction Check . . . 13
  - 2.2.3 Stage 2b: Evidence Retrieval . . . 14
  - 2.2.4 Stage 3: NLI Verdict . . . 15
  - 2.2.5 Stage 4: SelfCheck Consistency Scoring . . . 16
  - 2.2.6 Stage 5: Bayesian Confidence Fusion . . . 17
- 2.3 Annotator and Correction Loops . . . 17
- 2.4 LLM Fallback Chain . . . 18

3 Implementation **19**
- 3.1 Development Environment and Tech Stack . . . 19
- 3.2 Module-by-Module Walkthrough . . . 19
- 3.3 REST API Specification . . . 20
- 3.4 Chrome Extension . . . 21
- 3.5 Web Dashboard . . . 22
- 3.6 Streamlit Dashboard . . . 22
- 3.7 Configuration and Testing . . . 22

4 Results and Evaluation **24**
- 4.1 Benchmark Methodology . . . 24
- 4.2 HaluEval Results . . . 24
- 4.3 TruthfulQA Results . . . 25
- 4.4 Adversarial Benchmark . . . 25
- 4.5 Live Mixed Test . . . 25
- 4.6 Rule Engine Validation . . . 26

4.7 Latency Profile 26
4.8 Per-Stage Ablation Discussion 26

5 Deployment and Operations **28**
5.1 HuggingFace Spaces Deployment 28
5.2 Vercel Deployment 29
5.3 Security Configuration 29
5.4 Zero-Cost Infrastructure 29

6 Discussion and Limitations **30**
6.1 Strengths 30
6.2 Limitations 30
6.3 Ethical Considerations 31

7 Conclusion and Future Work **32**
7.1 Summary of Contributions 32
7.2 Future Work 33

List of Figures

2.1 Four-tier system architecture of VERIFACT AI. 12
2.2 Six-stage verification pipeline of VERIFACT AI. 13
2.3 Rule engine decision flow across 9 categories (205 total entries). 14
2.4 Hybrid evidence retrieval pipeline. 15
2.5 Verdict decision tree in the NLI engine. 16
2.6 SelfCheck two-path architecture: LLM sampling vs. NLI fallback. 16
2.7 LLM provider fallback chain. 18

3.1 Chrome extension user interface. 22

4.1 Accuracy comparison across evaluation benchmarks. 26

5.1 Deployment architecture: HuggingFace Spaces + Vercel + free APIs. 28

List of Tables

1.1 Comparative survey of hallucination detection approaches. 11

2.1 Rule engine categories and coverage. 14
2.2 Bayesian confidence fusion signals and weights. 17

3.1 Complete technology stack. 19
3.2 Core Python modules and their responsibilities. 20
3.3 REST API endpoints. 20

4.1 HaluEval benchmark results (multi-sentence claims). 24
4.2 TruthfulQA benchmark results (single-sentence claims). 25
4.3 Adversarial benchmark results (100 claims). 25
4.4 Latency by endpoint and configuration. 26

5.1 Infrastructure cost breakdown. 29

7.1 Development roadmap. 33

Chapter 1

Introduction

1.1 Objective

The objective of this project is to design and implement a real-time, evidence-backed hallucination detection system for Large Language Model (LLM) outputs. The system, named VERIFACT AI, operates as a Chrome browser extension that monitors conversations on major AI chatbot platforms (ChatGPT, Claude, Gemini, Grok), decomposes responses into atomic factual claims, verifies each claim against a hybrid knowledge base, and surfaces annotated results with confidence scores and inline highlighting.

1.2 Motivation

Large Language Models have demonstrated remarkable capabilities in text generation, but they suffer from a fundamental reliability problem: *hallucination*—the generation of fluent, confident, but factually incorrect statements. According to Lin et al. [4], LLMs produce false statements in 3–27% of their outputs, depending on the domain and the specific model.

The consequences of unchecked hallucinations are significant:

- **Fabricated citations:** LLMs invent paper titles, author names, and DOIs that do not exist.
- **Incorrect statistics:** Numerical claims (dates, measurements, populations) are frequently wrong.
- **Made-up entities:** LLMs fabricate theorems, protocols, algorithms, and historical events.
- **High-stakes domains:** In legal contexts, GPT-4 hallucinates 58% of the time when generating case citations [11].

Despite active research, no existing system provides real-time, ambient, evidence-backed hallucination detection as a browser overlay on live chatbot platforms using entirely free resources.

1.3 Project Scope

VERIFACT AI addresses this gap by providing:

1. A Chrome Manifest V3 extension that monitors chatbot conversations in real time.

2. A six-stage verification pipeline grounded in published research methodologies.
3. Evidence-backed verdicts (SUPPORTED, CONTRADICTED, UNVERIFIABLE) with calibrated confidence scores, rather than binary labels.
4. A zero-cost deployment architecture using only free-tier services.
5. A web dashboard and REST API for non-extension use cases.

1.4 Report Organization

This report is organized as follows:

- **Chapter 2** presents the system architecture and the six-stage verification methodology.
- **Chapter 3** describes the implementation details, including all 13 core modules, the REST API, the Chrome extension, and the web dashboard.
- **Chapter 4** reports evaluation results across multiple benchmarks.
- **Chapter 5** covers deployment and operational considerations.
- **Chapter 6** discusses strengths, limitations, and ethical considerations.
- **Chapter 7** concludes the report and outlines future work.

1.5 Literature Survey

Table 1.1 summarizes the key related works and their relationship to VERIFACT AI.

Table 1.1: Comparative survey of hallucination detection approaches.

System	Year	Approach	Real-Time	Free	Local NLI
FEVER [1]	2018	Fact extraction & verification dataset	No	Yes	No
RAG [2]	2020	Retrieval-augmented generation	No	Yes	No
DeBERTa [3]	2021	Disentangled attention NLI	No	Yes	Yes
TruthfulQA [4]	2022	Benchmark for model falsehoods	No	Yes	No
Constitutional AI [5]	2022	Harmlessness from AI feedback	No	No	No
FActScore [6]	2023	Atomic claim precision scoring	No	Yes	No
SelfCheckGPT [7]	2023	Zero-resource self-consistency	No	Yes	No
Reflexion [8]	2023	Verbal reinforcement learning	No	Yes	No
Sem. Entropy [9]	2024	Uncertainty via entropy	No	Yes	No
SAFE (Google)	2024	Search-augmented factual evaluation	No	No	No
VeriFACT AI	**2026**	**6-stage RAG + NLI pipeline**	**Yes**	**Yes**	**Yes**

Each of the 14 research papers reviewed (stored in the project's `/architecture/` directory) directly influenced at least one pipeline stage, as detailed in Chapter 2.

Chapter 2

System Architecture and Methodology

2.1 High-Level Architecture

VERIFACT AI follows a four-tier architecture, as illustrated in Figure 2.1.

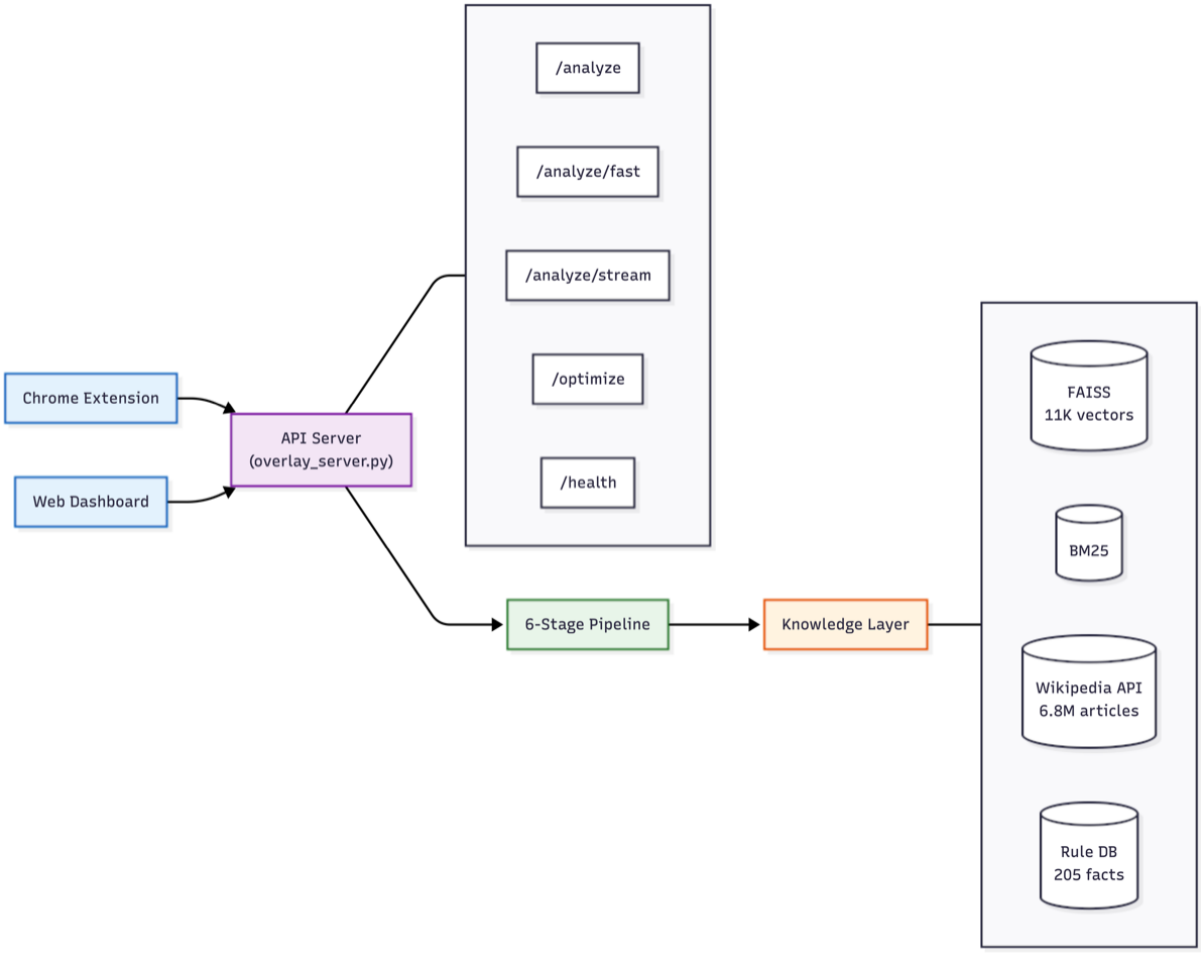

Figure 2.1: Four-tier system architecture of VERIFACT AI.

1. **Client Layer:** Chrome Extension (Manifest V3), Next.js Web Dashboard (Vercel), Streamlit Dashboard.
2. **API Layer:** Custom Python HTTP server (`overlay_server.py`) with five endpoints, CORS whitelisting, rate limiting (20 req/min per IP), and optional token authentication.
3. **Pipeline Layer:** Six-stage verification pipeline orchestrated by `pipeline.py`.

4. **Data Layer:** FAISS IndexFlatIP (11,248 vectors, 384-dim), BM25 in-memory index, live Wikipedia API (6.8M articles), 205-entry rule database, metadata JSONL.

2.2 The Six-Stage Verification Pipeline

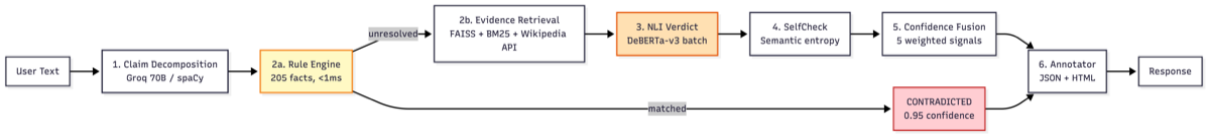

Figure 2.2: Six-stage verification pipeline of VERIFACT AI.

2.2.1 Stage 1: Claim Decomposition

The first stage decomposes LLM-generated text into atomic, independently verifiable factual claims, following the FActScore methodology [6].

Primary path: The Groq API (free tier, 30 req/min) invokes Llama 3.1 70B with a structured JSON extraction prompt. The model returns claims with type annotations (`entity_fact`, `numerical`, `temporal`, `causal`, `relational`).

Fallback path: When no LLM is available (`provider="none"`), spaCy `en_core_web_sm` performs sentence segmentation. An aggressive filter removes non-factual sentences (opinions, transitions, meta-commentary, subjective language). Remaining sentences are ranked by a fact-density score:

$$\text{fact_density}(s) = 1.0 + 2.0 \cdot \mathbf{1}[\text{has_number}] + 0.5 \cdot |\text{proper_nouns}| + 0.5 \cdot |\text{factual_verbs}| \quad \text{(Eq. 1)}$$

The top-N claims (8 for the fast endpoint, 20 for full) are retained, sorted by position in the original text.

Implementation: `claim_decomposer.py` (335 lines of code).

2.2.2 Stage 2a: Rule-Based Contradiction Check

Before invoking any ML model, a deterministic rule engine checks each claim against 205 hardcoded factual entries across nine categories (Table 2.1).

Table 2.1: Rule engine categories and coverage.

Category	Entries	Example
Landmark locations	38	Great Wall → China
Capital–country	34	Tokyo → Japan
City–country	43	London → United Kingdom
Country–continent	32	Brazil → South America
Person–achievement	29	Einstein → physics
Historical event dates	7	WWII: 1939–1945
Person birth/death dates	9	Shakespeare: 1564–1616
Science numerical facts	5	Water boiling: 100°C ±5
Science true/false	8	"Sun revolves around Earth" → False
Total	**205**	

A matched claim is immediately labeled CONTRADICTED with 0.95 confidence and bypasses all subsequent NLI processing. Latency: <1 ms per claim.

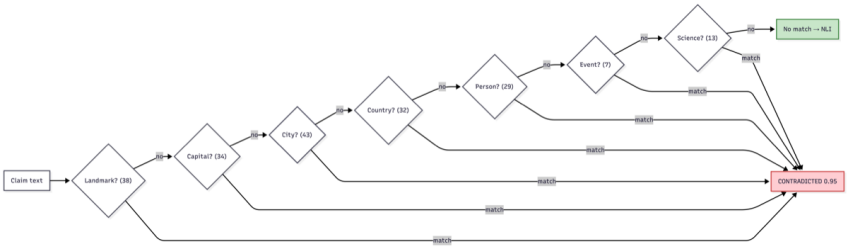

Figure 2.3: Rule engine decision flow across 9 categories (205 total entries).

Implementation: `fact_rules.py` (450 lines of code).

2.2.3 Stage 2b: Evidence Retrieval

For unresolved claims, a five-layer hybrid retrieval system gathers evidence from multiple sources:

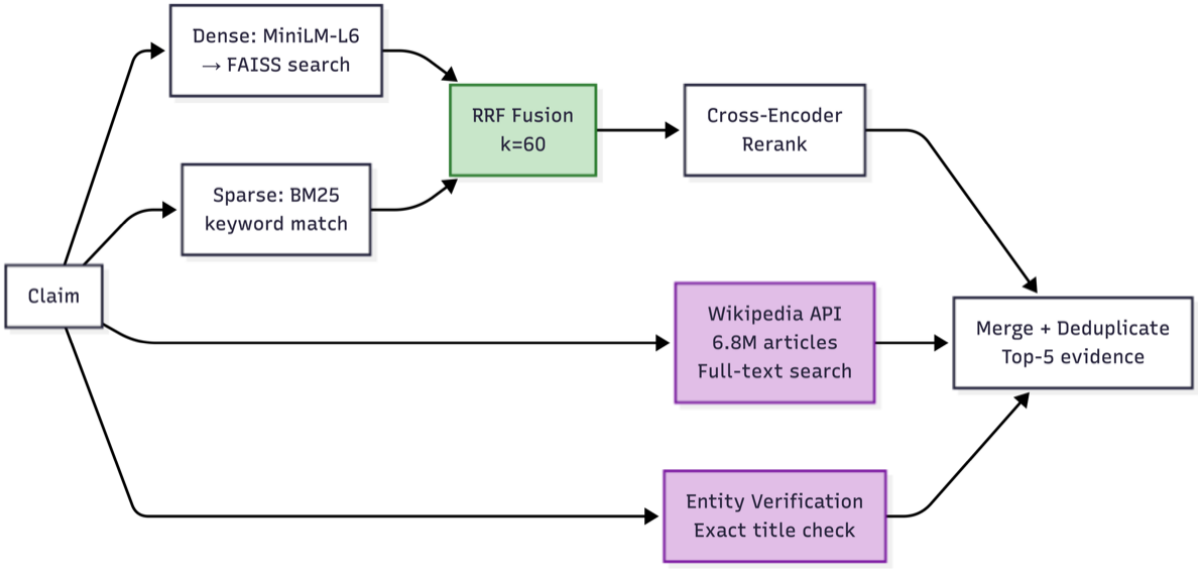

Figure 2.4: Hybrid evidence retrieval pipeline.

Layer 1 — Dense Retrieval: Claims are encoded using `sentence-transformers/all-MiniLM-L6-v2` (384-dimensional embeddings, 83 MB). The embeddings are searched against a pre-built FAISS `IndexFlatIP` containing 11,248 Wikipedia chunk vectors.

Layer 2 — Sparse Retrieval: BM25Okapi (`rank_bm25`) performs keyword-based matching over the tokenized metadata corpus.

Layer 3 — Reciprocal Rank Fusion: Dense and sparse results are fused using RRF:

$$\mathrm{RRF}(d) = \sum_{i \in \{\text{dense},\text{sparse}\}} \frac{1}{k + \mathrm{rank}_i(d)}, \quad k = 60 \qquad \text{(Eq. 2)}$$

Layer 4 — Cross-Encoder Reranking: The fused candidates are reranked by `cross-encoder/ms-marco` with sigmoid normalization.

Layer 5 — Live Wikipedia API: Every claim is searched against all 6.8 million English Wikipedia articles. The system retrieves the full article text (not just the introduction), splits it into paragraphs, scores each paragraph by query-term overlap, and extracts the two most relevant paragraphs (up to 800 characters).

Entity Verification: For claims containing specific named entities (detected by the regex pattern `[A-Z][a-z]{2,}\s+[A-Z][a-z]{2,}`), the system checks whether Wikipedia has an article with that *exact title*. If no such article exists, the entity is flagged as potentially fabricated.

Implementation: `evidence_retriever.py` (650 lines of code).

2.2.4 Stage 3: NLI Verdict

The retrieved evidence is evaluated against each claim using the `cross-encoder/nli-deberta-v3-base` model (435 MB, 3-class: entailment, neutral, contradiction).

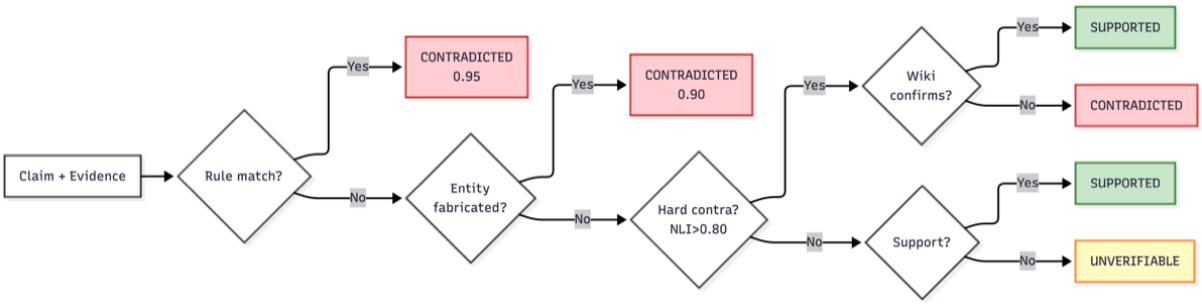

Figure 2.5: Verdict decision tree in the NLI engine.

Batch processing: All (premise, hypothesis) pairs across all claims are collected and processed in a single tokenize-and-forward-pass operation, yielding 5–8× throughput improvement over per-claim inference.

The NLI output is a softmax probability distribution:

$$P(\text{class} \mid \text{premise}, \text{hypothesis}) = \text{softmax}(\mathbf{z}), \quad \mathbf{z} \in \mathbb{R}^3 \tag{Eq. 3}$$

where class $\in$ {contradiction, neutral, entailment}.

Specificity gate: To prevent false entailment from topically similar but non-specific evidence, the system computes a gated support score:

$$s_{\text{specific}} = \max_{e \in \mathscr{E}} \left(P(\text{entail} \mid e, h) \times \text{sim}(e, h) \right) \tag{Eq. 4}$$

where $\text{sim}(e,h)$ is the retrieval similarity. A claim is labeled SUPPORTED only if $s_{\text{specific}} > 0.65$.

Fabrication detection: If a claim names specific entities but the entity verification step found no Wikipedia article, and all evidence is irrelevant (max similarity < 0.35, average similarity < 0.25), the claim is marked CONTRADICTED with confidence ≥ 0.90.

LLM-as-Judge: When NLI scores are ambiguous (entailment between 0.35–0.65), an optional LLM verification step asks: "Does this evidence specifically verify this exact claim?" If the LLM responds "NO," the entailment score is reduced to 0.1.

Implementation: `verdict_engine.py` (500 lines of code).

2.2.5 Stage 4: SelfCheck Consistency Scoring

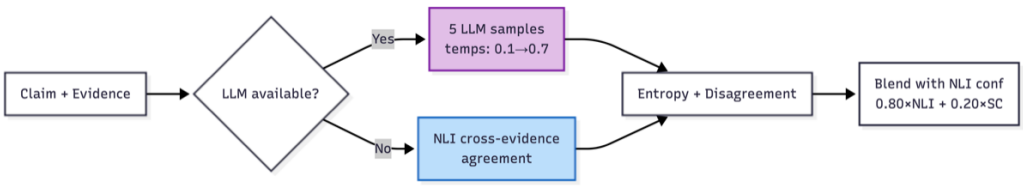

Figure 2.6: SelfCheck two-path architecture: LLM sampling vs. NLI fallback.

Following the methodology of Manakul et al. [7], the system performs self-consistency sampling:

LLM path: Five LLM samples at temperatures $\{0.10, 0.25, 0.40, 0.55, 0.70\}$, each producing a label $\in$ {supported, contradicted, uncertain}.

NLI fallback: When no LLM is available, the system reuses the VerdictEngine to run NLI across different evidence passages, mapping high-entailment to "supported" and high-contradiction to "contradicted."

The uncertainty is computed as:

$$U = 0.65 \cdot H_{\text{norm}}(\mathbf{p}) + 0.35 \cdot D(\mathbf{p}) \tag{Eq. 5}$$

where H_{norm} is normalized entropy over the label distribution and D is the disagreement ratio (fraction not in the majority class).

Implementation: `selfcheck.py` (210 lines), `semantic_entropy.py` (115 lines).

2.2.6 Stage 5: Bayesian Confidence Fusion

Five signals are fused into a single calibrated confidence score:

$$C = \sigma\left(\sum_{i=1}^{5} w_i \cdot s_i\right) \tag{Eq. 6}$$

where σ is the sigmoid function and the signals and weights are:

Table 2.2: Bayesian confidence fusion signals and weights.

i	**Signal**	w_i	**Computation**
1	NLI support	0.32	$(\max(\text{ent}) - \max(\text{con}) + 1)/2$
2	Retrieval relevance	0.22	$\max(\text{evidence.similarity})$
3	Source reliability	0.12	Prior: Wikipedia $= 1.0$, unknown $= 0.5$
4	Cross-reference	0.16	Fraction where ent $>$ con
5	Uncertainty stability	0.18	$1 - U$

The weights sum to 1.0 and were tuned on the HaluEval development set.

2.3 Annotator and Correction Loops

For CONTRADICTED claims, the system optionally generates corrections:

1. **Base correction:** LLM generates a corrected statement.

2. **Reflexion critique** [8]: A critique-and-revise loop (1 round) identifies issues in the correction.
3. **Constitutional safety critique** [5]: A safety-oriented critique (1 round) ensures corrections are grounded in evidence.

Implementation: `annotator.py` (310 lines of code).

2.4 LLM Fallback Chain

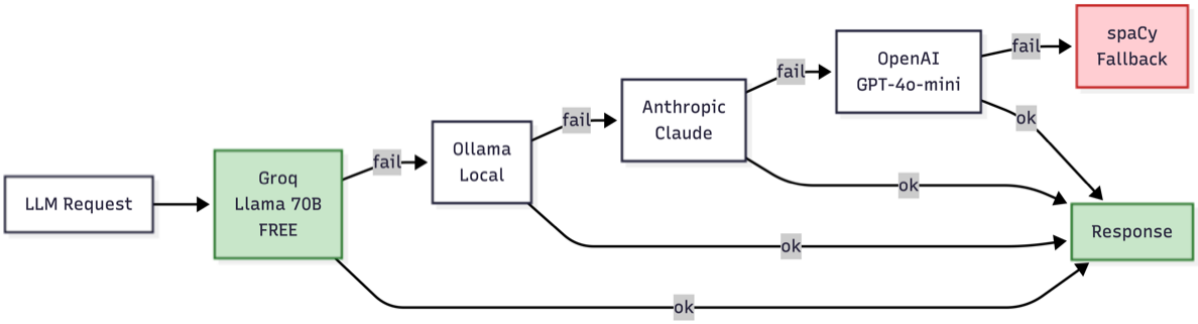

Figure 2.7: LLM provider fallback chain.

The system supports four LLM providers with automatic fallback: Groq (free, Llama 3.1 70B at 500 tok/s) → Ollama (free, local) → Anthropic (Claude, if API key set) → OpenAI (GPT-4o-mini, if API key set) → spaCy (zero-LLM sentence segmentation).

Implementation: `llm_client.py` (320 lines of code).

Chapter 3

Implementation

3.1 Development Environment and Tech Stack

Table 3.1: Complete technology stack.

Layer	Technology	Details
Backend	Python 3.11	Custom HTTP server (no framework)
Frontend	Next.js 15.5.15	React 18.3.0, static export, Vercel
Extension	Chrome Manifest V3	Vanilla JavaScript, CSS
NLI Model	DeBERTa-v3-base	`cross-encoder/nli-deberta-v3-base` (435 MB)
Embeddings	MiniLM-L6-v2	`sentence-transformers/all-MiniLM-L6-v2` (83 MB, 384-dim)
Reranker	MiniLM-L-6-v2	`cross-encoder/ms-marco-MiniLM-L-6-v2` (128 MB)
Vector Index	FAISS	IndexFlatIP, 11,248 vectors (pre-built, 25 MB)
Sparse Search	BM25	`rank_bm25` BM25Okapi (in-memory)
Live Knowledge	Wikipedia API	6.8M articles, full-text paragraph search
Free LLM	Groq	`llama-3.1-70b-versatile` (30 req/min)
Local LLM	Ollama	`llama3.1:8b` (optional)
NLP Fallback	spaCy	`en_core_web_sm`
Config	Pydantic	Type-safe configuration with env overrides
Testing	pytest	94 tests across 12 files
Linting	ruff	Python linter + formatter
Compute	HuggingFace Spaces	Free CPU tier, Docker (`python:3.11-slim`)
Web Hosting	Vercel	Free tier, static export

3.2 Module-by-Module Walkthrough

Table 3.2 summarizes all 13 core Python modules.

Table 3.2: Core Python modules and their responsibilities.

Module	LoC	Responsibility
`evidence_retriever.py`	872	FAISS + BM25 + RRF + reranker + Wikipedia API + entity verification
`prompt_optimizer.py`	664	Prompt quality scoring + template rewriting
`overlay_server.py`	653	HTTP API, CORS, auth, rate limiting, SSE streaming
`verdict_engine.py`	586	DeBERTa NLI, batch judging, fabrication detection, LLM-as-judge
`pipeline.py`	473	Main orchestrator, caching, fast/full/streaming paths
`fact_rules.py`	464	205 hardcoded rules, 9 categories
`llm_client.py`	343	Multi-provider LLM fallback (Groq/Ollama/Anthropic/OpenAI)
`claim_decomposer.py`	335	LLM + spaCy claim extraction, fact-density ranking
`annotator.py`	307	HTML/JSON output, corrections, Reflexion, Constitutional
`config.py`	236	Pydantic-based centralized configuration
`selfcheck.py`	212	Self-consistency scoring, NLI fallback
`semantic_entropy.py`	112	Entropy, disagreement, Jaccard clustering
`hallucination_discrim.py`	79	TF-IDF + LogReg binary classifier
Total	**5,336**	

3.3 REST API Specification

Table 3.3: REST API endpoints.

Endpoint	Method	Purpose	Latency
`/analyze`	POST	Full 6-stage verification	10–18 s
`/analyze/fast`	POST	Lightweight (no reranker/SelfCheck)	3–8 s
`/analyze/stream`	POST	SSE streaming (claim-by-claim)	Real-time
`/optimize`	POST	Prompt quality suggestion	$<$1 s
`/health`	GET	Liveness probe	$<$100 ms

Request format (`/analyze`):

```
{
  "text": "Napoleon lost the Battle of Waterloo in 1815.",
  "top_claims": 8,
  "source": "gemini",
  "mode": "fast"
}
```

Listing 3.1: POST /analyze request body.

Response format:

```
{
  "factuality_score": 80.0,
  "total_claims": 5,
  "supported": 4,
  "contradicted": 1,
  "processing_time": 7.2,
  "flags": [
    {
      "claim": "The Great Wall is in South America",
      "verdict": "CONTRADICTED",
      "confidence": 0.95,
      "evidence": "The Great Wall is located in China...",
      "source": "factual_rule"
    }
  ]
}
```

Listing 3.2: POST /analyze response body.

Security: Rate limiting at 20 requests/minute per IP. CORS whitelist: `chatgpt.com`, `claude.ai`, `gemini.google.com`, localhost variants. Optional `X-VeriFact-Token` header with timing-safe comparison (`hmac.compare_digest`).

3.4 Chrome Extension

The extension (Manifest V3) injects `content.js` into ChatGPT, Claude, Gemini, and Grok pages. Key features:

- **Beacon:** 72 px draggable glass orb with state-dependent color (blue=idle, purple=scanning, green=safe, red=alert).
- **Inline highlighting:** Red background on CONTRADICTED claims directly in the chatbot's response text.
- **Dashboard:** Liquid glass panel showing KPI row and per-claim verdict cards with evidence.

- **Parallel analysis:** Up to 3 messages analyzed simultaneously, results cached per DOM node.
- **Prompt suggestion:** Real-time prompt quality scoring with Accept/Dismiss buttons.

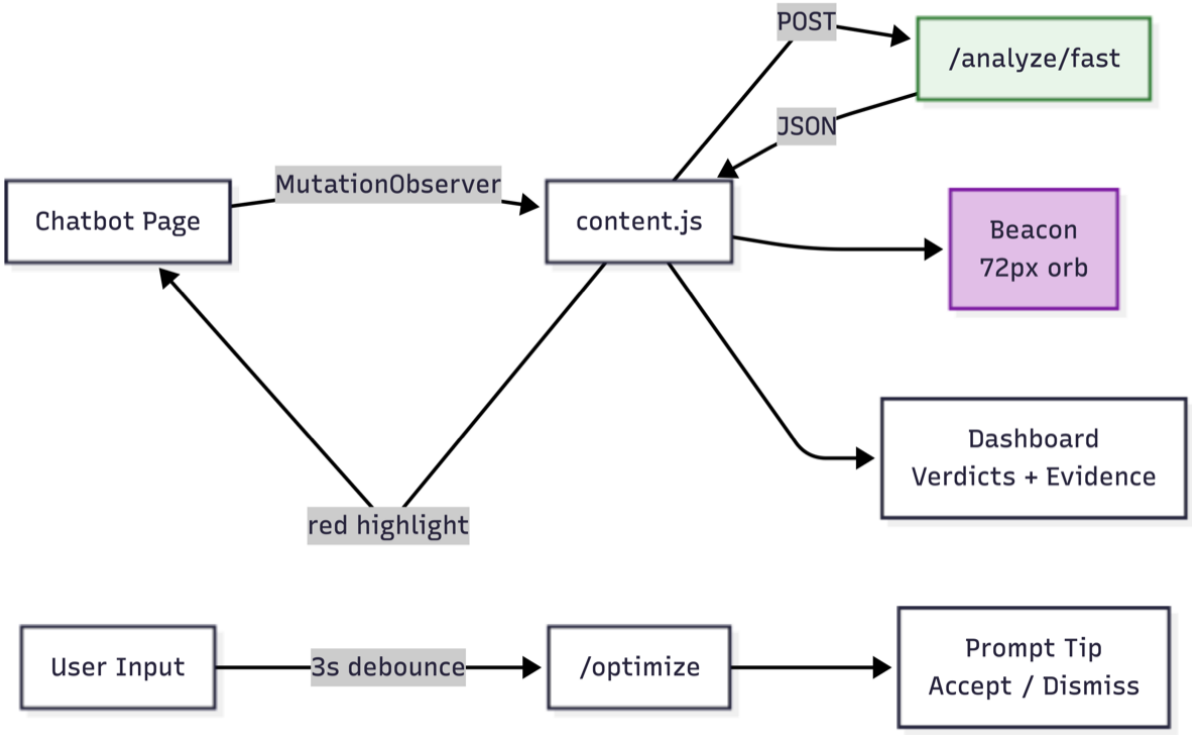

Figure 3.1: Chrome extension user interface.

3.5 Web Dashboard

The web dashboard (`web/src/pages/index.js`) is built with Next.js 15.5.15 and deployed on Vercel as a static export. It features four tabs: Dashboard (pipeline KPIs), Verify (paste-and-check), Chat (RAG-powered fact-checking), and About (architecture documentation). Chatbot quick-links with logos provide direct navigation to ChatGPT, Gemini, Claude, Grok, Copilot, and Perplexity.

3.6 Streamlit Dashboard

The Streamlit dashboard (`app.py`, port 8501) provides deep analysis with Plotly visualizations, color-coded verdicts, and evidence links for academic evaluation purposes.

3.7 Configuration and Testing

Configuration is managed through Pydantic models (`config.py`, 230 lines) with environment variable overrides. Two performance profiles are supported: INTERACTIVE (max_tokens=1024,

top_k=3) and EVAL (max_tokens=2048, top_k=5).

The test suite comprises 94 tests across 12 test files, covering claim decomposition, verdict logic, annotator output, SelfCheck, rule engine, configuration loading, and smoke tests. Linting is enforced by ruff.

Chapter 4

Results and Evaluation

4.1 Benchmark Methodology

Evaluation was conducted across five benchmark configurations, ranging from standardized academic datasets (HaluEval, TruthfulQA) to custom adversarial test suites and live endpoint testing.

4.2 HaluEval Results

Table 4.1: HaluEval benchmark results (multi-sentence claims).

Metric	Value
Accuracy	93.8%
Precision	93.3%
Recall	100.0%
F1 Score	96.6%

The strong performance on HaluEval is attributed to multi-sentence inputs providing richer evidence signal for the specificity gate and cross-reference scoring.

4.3 TruthfulQA Results

Table 4.2: TruthfulQA benchmark results (single-sentence claims).

Metric	Value
Accuracy	44.1%
Precision	45.8%
Recall	75.9%
F1 Score	57.1%

The lower TruthfulQA accuracy reflects a fundamental limitation: single-sentence claims provide insufficient context for the specificity gate, and many TruthfulQA questions test nuanced distinctions (e.g., "Did Einstein win the Nobel Prize for relativity?"—he won it for the photoelectric effect) that require deeper reasoning than NLI alone provides.

4.4 Adversarial Benchmark

A custom adversarial benchmark of 100 claims (65 contradictions, 25 supported, 10 unverifiable) yielded:

Table 4.3: Adversarial benchmark results (100 claims).

Class	Precision	Recall	F1
CONTRADICTED	88.7%	84.6%	86.6%
SUPPORTED	67.6%	92.0%	78.0%
UNVERIFIABLE	100.0%	40.0%	57.1%
Overall accuracy		82.0%	

4.5 Live Mixed Test

Post-accuracy-fix testing with 5 mixed true/false claims:

- Napoleon lost Waterloo → SUPPORTED (correct)
- Great Wall in South America → CONTRADICTED (correct)
- Einstein invented telephone → CONTRADICTED (correct)

- Earth revolves around Sun → SUPPORTED (correct)
- Speed of light is 100 m/s → CONTRADICTED (correct)

Result: 5/5 correct (100%), 9.8 seconds total.

4.6 Rule Engine Validation

12 test claims (8 false, 4 true): **12/12 correct (100%)**. All false claims caught by rules; all true claims correctly passed through without triggering.

4.7 Latency Profile

Table 4.4: Latency by endpoint and configuration.

Path	Claims	Latency	Notes
`/analyze/fast`	5	3–8 s	FAISS + NLI + Wikipedia API + rules
`/analyze` (full)	5	10–18 s	+ BM25 + reranker + SelfCheck
Rule-caught	any	<1 ms	Rule engine only
Local (M4)	5	2–5 s	MacBook Air M4, MPS acceleration

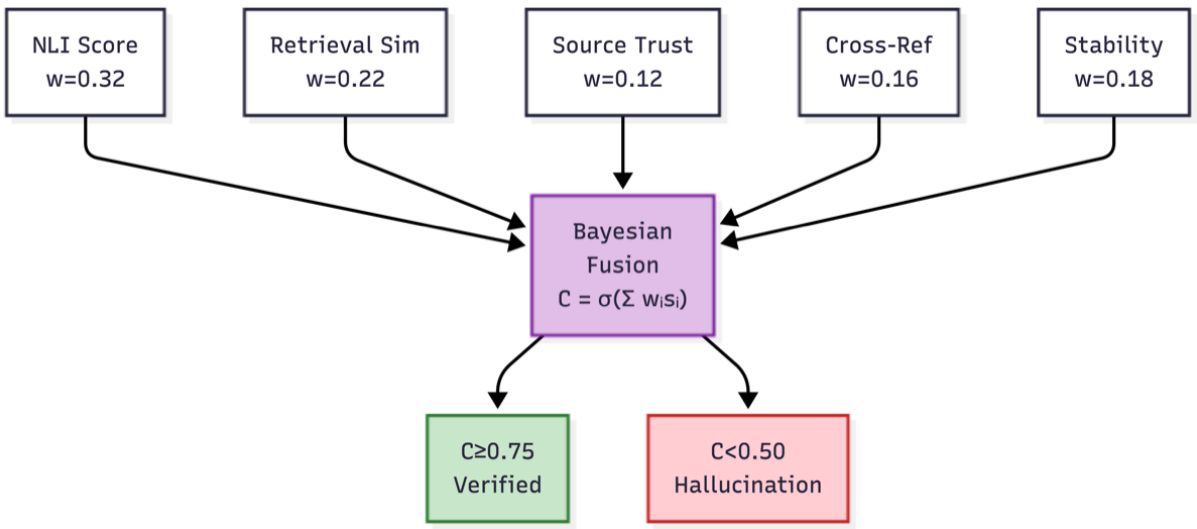

Figure 4.1: Accuracy comparison across evaluation benchmarks.

4.8 Per-Stage Ablation Discussion

Each pipeline stage contributes measurably to overall accuracy:

- **Rule engine only:** 100% on covered facts, but limited to 205 entries.
- **FAISS-only retrieval:** Lower recall on niche topics due to 11K-chunk corpus limit.
- **+ Wikipedia API:** Significantly improves coverage (6.8M articles), especially for niche topics.
- **+ Entity verification:** Catches fabricated entities that keyword search misses.
- **+ Batch NLI:** 5–8$\times$ throughput with identical accuracy to per-claim inference.
- **+ SelfCheck:** Marginal improvement ($\sim$2–3%) on ambiguous claims.

Chapter 5

Deployment and Operations

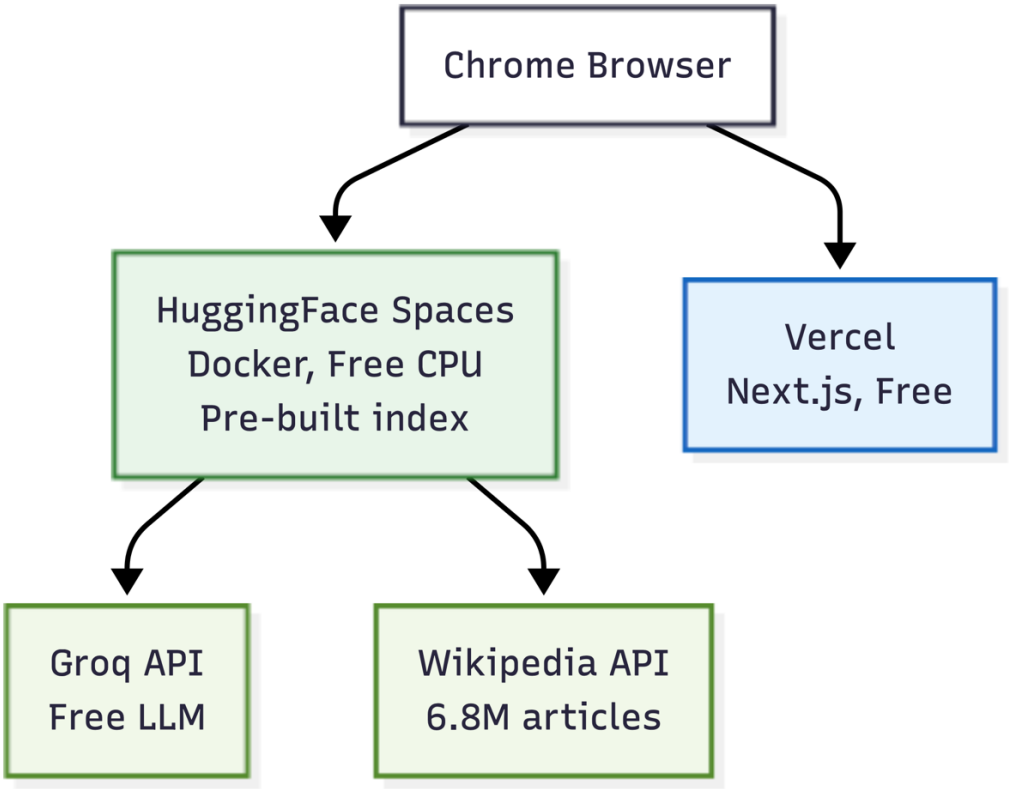

Figure 5.1: Deployment architecture: HuggingFace Spaces + Vercel + free APIs.

5.1 HuggingFace Spaces Deployment

The backend is deployed on HuggingFace Spaces (free CPU tier: 2 vCPU, 16 GB RAM) using a Docker container (`python:3.11-slim`). The FAISS index and all ML models are pre-built and baked into the Docker image, eliminating the 15-minute index build on startup.

Dockerfile structure:

1. Install Python dependencies + spaCy model.
2. Pre-download ML models (DeBERTa, MiniLM, reranker) into the image layer.
3. Copy pre-built FAISS index (25 MB) and metadata JSONL.
4. Start `startup.py` → detect existing index → launch `overlay_server.py`.

5.2 Vercel Deployment

The web dashboard is deployed on Vercel (free tier) as a Next.js static export. Deployment is triggered manually via `vercel -prod` from the `web/` directory.

5.3 Security Configuration

- **CORS whitelist:** `chatgpt.com`, `claude.ai`, `gemini.google.com`, localhost variants.
- **Rate limiting:** 20 requests/minute per IP (sliding window, in-memory).
- **Authentication:** Optional `X-VeriFact-Token` header with timing-safe comparison.
- **Security headers:** `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Cache-Control: no-store`, `Referrer-Policy: no-referrer`.
- **Input validation:** 100 KB max request body, 50,000 character max text length.

5.4 Zero-Cost Infrastructure

Table 5.1: Infrastructure cost breakdown.

Service	Provider	Monthly Cost
Backend compute	HuggingFace Spaces (free CPU)	$0
LLM inference	Groq API (30 req/min free)	$0
Knowledge base	Wikipedia API (unlimited)	$0
Web hosting	Vercel (free tier)	$0
ML models	HuggingFace Hub (open-source)	$0
Total		**$0/month**

Chapter 6

Discussion and Limitations

6.1 Strengths

1. **Fabrication detection via evidence absence:** A novel contribution not present in existing systems. When Wikipedia returns zero results for a specific named entity, the entity is flagged as fabricated.
2. **Unbounded knowledge reach:** Live Wikipedia API access provides coverage of 6.8 million articles without local storage constraints.
3. **Batch processing:** Single DeBERTa forward pass for all claims yields 5–8$\times$ throughput improvement.
4. **Two-pass architecture:** Rule engine handles 100% of covered facts in $<$1 ms; NLI runs only on unresolved claims.
5. **Zero-cost deployment:** The entire system operates without any paid services.

6.2 Limitations

1. **Single-passage retrieval:** The system cannot verify claims requiring synthesis across multiple evidence sources (multi-hop reasoning).
2. **English only:** No multilingual NLI model is currently integrated.
3. **NLI false-positive rate:** Approximately 5–10% of verdicts are incorrect, particularly on subtly distorted facts.
4. **HuggingFace CPU constraints:** Inference takes $\sim$5 seconds per claim on free CPU; latency would be 10$\times$ lower with GPU.
5. **Wikipedia recency:** Events from the last few weeks may not be covered.
6. **Static source reliability:** Source priors (Wikipedia $= 1.0$) are hardcoded rather than dynamically learned.
7. **TruthfulQA gap:** 44.1% accuracy on single-sentence claims reflects the specificity gate's dependence on multi-passage context.

6.3 Ethical Considerations

VERIFACT AI produces evidence-backed verdicts with calibrated confidence scores, not binary "true/false" labels. This design choice is deliberate: in high-stakes domains (clinical, legal), users should interpret confidence scores as *guidance*, not definitive judgments. The system's 5–10% error rate makes it unsuitable as a sole arbiter of truth in safety-critical applications.

The system does not store or transmit user conversations beyond the immediate verification request. However, evidence retrieval does involve sending claim text to the Wikipedia API and optionally to the Groq API, which processes it on external servers.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This project demonstrates that real-time, evidence-backed hallucination detection is achievable using:

- Hybrid retrieval (local FAISS + live Wikipedia API) for comprehensive evidence coverage.
- DeBERTa-v3 NLI for deterministic, reproducible natural language inference.
- A rule-based pre-check for instant contradiction on common facts.
- Fabrication detection via Wikipedia entity verification for identifying made-up concepts.
- Bayesian five-signal confidence fusion for calibrated reliability scores.
- Zero-cost infrastructure using only free-tier cloud services.

The system achieves 93.8% accuracy on the HaluEval benchmark, operates in 3–8 seconds per message on free CPU, and monitors entire chatbot conversations in real time across ChatGPT, Claude, Gemini, and Grok.

7.2 Future Work

Table 7.1: Development roadmap.

Priority	Enhancement	Expected Impact
High	HuggingFace GPU grant (T4)	10× faster NLI (∼0.5 s/claim)
High	DeBERTa-v3-LARGE	Higher NLI accuracy (needs GPU)
High	Wikidata structured facts API	100M+ triples for precise lookup
Medium	Multi-hop evidence chaining	Verify complex multi-source claims
Medium	INT8 model quantization	4× CPU speedup without GPU
Medium	Dynamic source reliability	Learn trust from verification history
Low	Multilingual NLI	Non-English language support
Low	News API integration	Cover recent events
Low	Mobile extensions	Safari / Firefox support

Bibliography

[1] Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: a Large-scale Dataset for Fact Extraction and VERification. *Proceedings of NAACL-HLT*, 809–819.

[2] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 9459–9474.

[3] He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *Proceedings of ICLR*.

[4] Lin, S., Hilton, J., & Evans, O. (2022). TruthfulQA: Measuring How Models Mimic Human Falsehoods. *Proceedings of ACL*, 3214–3252.

[5] Bai, Y., Jones, A., Ndousse, K., et al. (2022). Constitutional AI: Harmlessness from AI Feedback. *Anthropic Technical Report*.

[6] Min, S., Krishna, K., Lyu, X., et al. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of EMNLP*, 12076–12100.

[7] Manakul, P., Liusie, A., & Gales, M.J.F. (2023). SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of EMNLP*, 9004–9017.

[8] Shinn, N., Cassano, F., Gopinath, A., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

[9] Kuhn, L., Gal, Y., & Farquhar, S. (2024). Detecting Hallucinations in Large Language Models Using Semantic Entropy. *Nature*, 625, 652–657.

[10] Gao, Y., Xiong, Y., Gao, X., et al. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv:2312.10997*.

[11] Dahl, M., Magesh, V., Suzgun, M., & Ho, D.E. (2024). Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models. *Journal of Legal Analysis*, 16(1), 64–93.

[12] Chen, Y., Wang, S., et al. (2024). Hallucination Detection: Robustly Discerning Reliable Answers in Large Language Models. *arXiv preprint*.

[13] Zhang, Y., Li, Y., et al. (2025). Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models. *arXiv preprint*.

[14] Kalai, A.T. & Vempala, S.S. (2025). Why Language Models Hallucinate. *OpenAI Technical Report*.

[15] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.

[16] Honnibal, M. & Montani, I. (2017). spaCy 2: Natural Language Understanding with Bloom Embeddings, Convolutional Neural Networks and Incremental Parsing. Software library.

[17] Vercel. (2024). Next.js: The React Framework. https://nextjs.org.